Smart Traffic Light Controller with Emergency Vehicle Preemption

Course: COSC-3407-W05 Operating Systems

Abstract

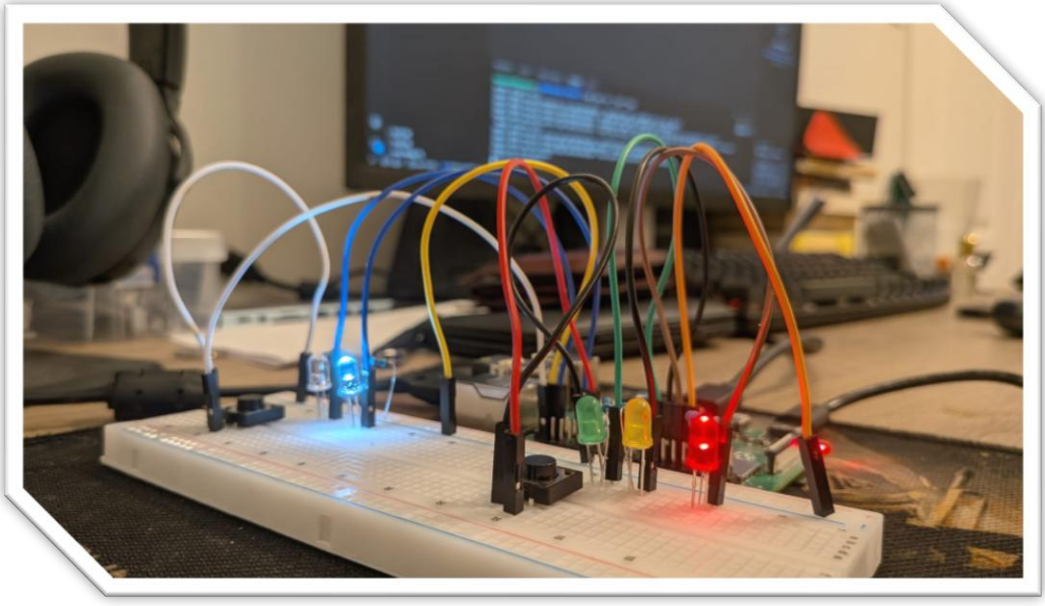
Our project demonstrates the application of pure Operating System concepts like multi threading, thread synchronization and priority scheduling through simulation of hardware. We built a smart traffic light system using a Raspberry Pi 4 coupled with the PI4J liabrary for Java. Hardware components like LED lights, pushbuttons, resistors and a breadboard were also used. Our system successfully managed three concurrent threads specifically considering a conventional traffic loop, a predestrian crossing monitor and an override for an emergency vehicle. Utilizing a semaphore for mutual exclusion ensured that no two threads could manipulate the traffic lights simultaneously which prevented the state conflicts of hardware and validated basic programming principles.

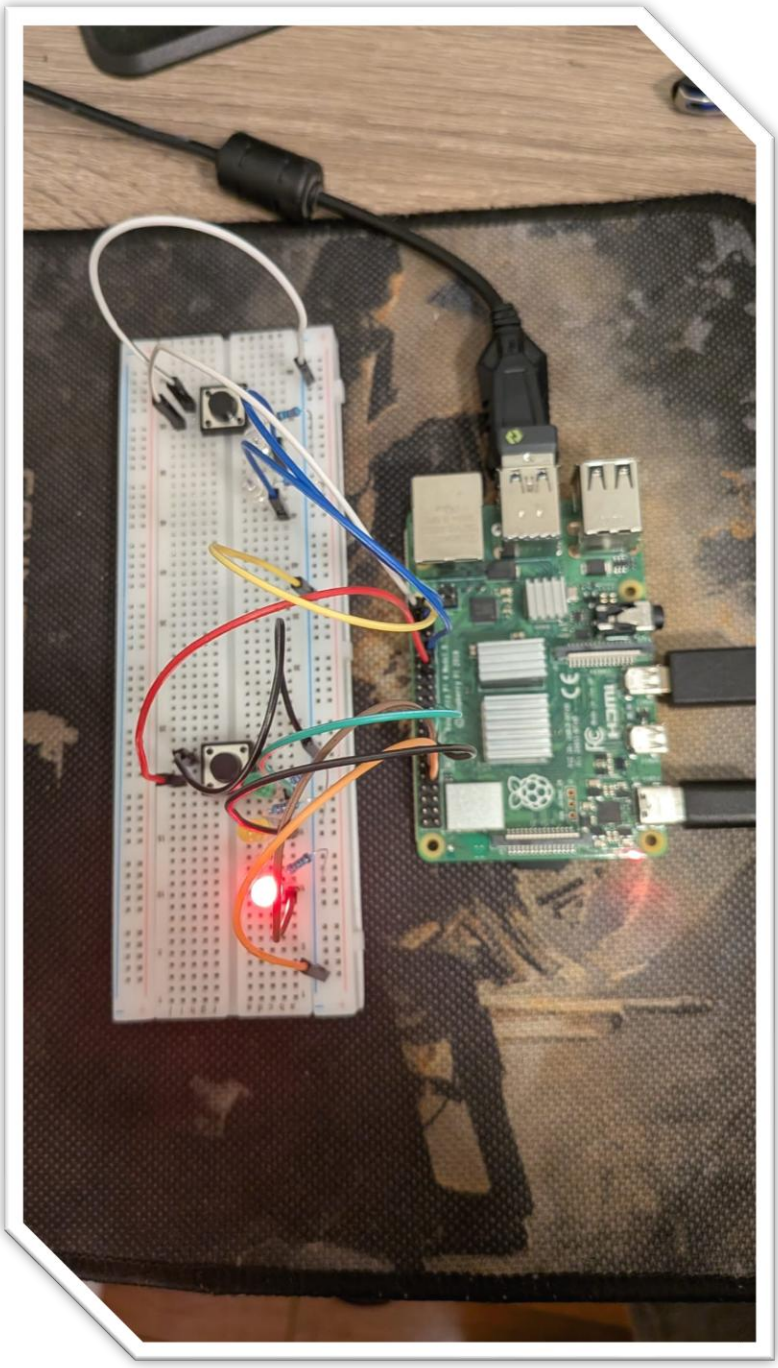
Introduction

Our objective of this project according to the project outline was to design and implement a multi threaded traffic light controller that handles multiple inputs without interrupting the safety and flow of the system. To prevent race in modern operating systems, handling multiple inputs requires strict resource management. Our project has simulated a modern intersection where an emergency vehicle must safely preempt normal traffic flow, which requires the system to stop its normal execution by securing shared resources and executing a higher priority routine.

Material and Methods

The simulation was constructed using a breadboard and a Raspberry Pi via GPIO pins. The hardware we used included one Red LED Light, 1 Yellow LED Light, 1 Green LED Light, 2 White LED Lights, two push buttons, 5 220 ohm resistors and a breadboard with jumper wires. According to what we studied in our previous courses our group members were fluent in Java so we used Java 17 and the PI4J library. This all was executed through a linux terminal.





Results and Code implementation

Our system successfully executed all required states according to the project outline and it didn't fall into a race condition.

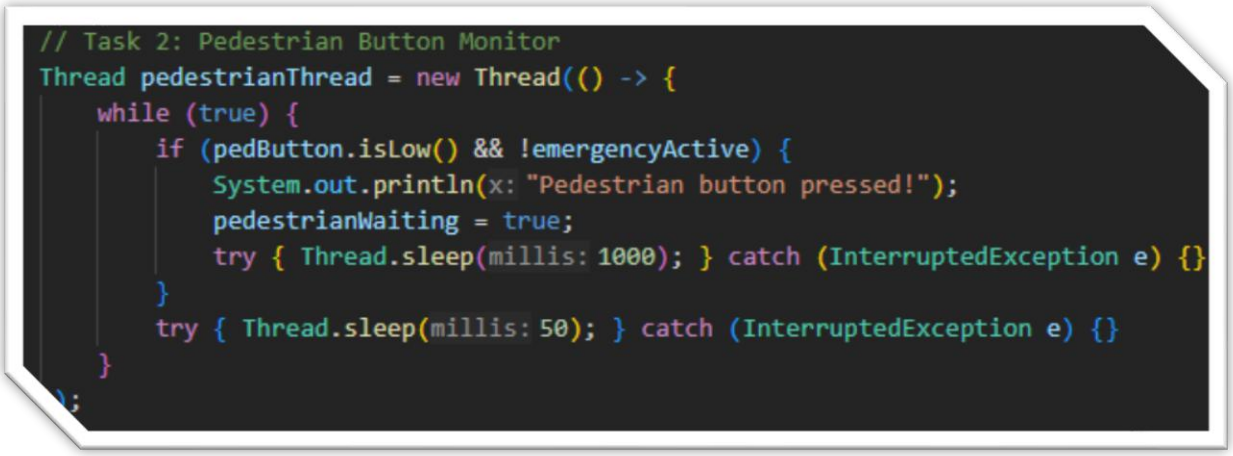
Task1: Normal Traffic Thread

This thread runs an infinite while loop which is while(true) it checks the emergencyActive variable which is also our shared variable before transitioning into any new state. if no emergency is detected it proceeds with the standard delay

```
// Task 1: Normal Traffic Cycle
Thread trafficThread = new Thread(() -> {
    try {
        while (true) {
            if (!emergencyActive) {
                lightMutex.acquire(); // Enter Critical Section

                // Green Light Cycle
                redLight.low();
                yellowLight.low();
                greenLight.high();
                pedLight.low();
                lightMutex.release(); // Leave Critical Section

                // Check if pedestrian is waiting during green light
                for(int i = 0; i < 50; i++) {
                    if(emergencyActive) break;
                    Thread.sleep(millis: 100);
                }

                if (emergencyActive) continue; // Skip to emergency

                lightMutex.acquire(); // Enter Critical Section

                // Yellow Light Cycle

                greenLight.low();
                yellowLight.high();
                lightMutex.release(); // Leave Critical Section
                Thread.sleep(millis: 2000);

                lightMutex.acquire(); // Enter Critical Section

                // Red Light Cycle

                yellowLight.low();
                redLight.high();

                // If pedestrian pushed button, let them walk during Red
                if (pedestrianWaiting) {
                    System.out.println(x: "Pedestrian is walking...");
                    pedLight.high();
                    Thread.sleep(millis: 4000);
                    pedLight.low();
                    pedestrianWaiting = false;
                } else {
                    Thread.sleep(millis: 3000);
                }
                lightMutex.release(); // Leave Critical Section
            }
            Thread.sleep(millis: 100);
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
```

Task2: Predestrian Thread

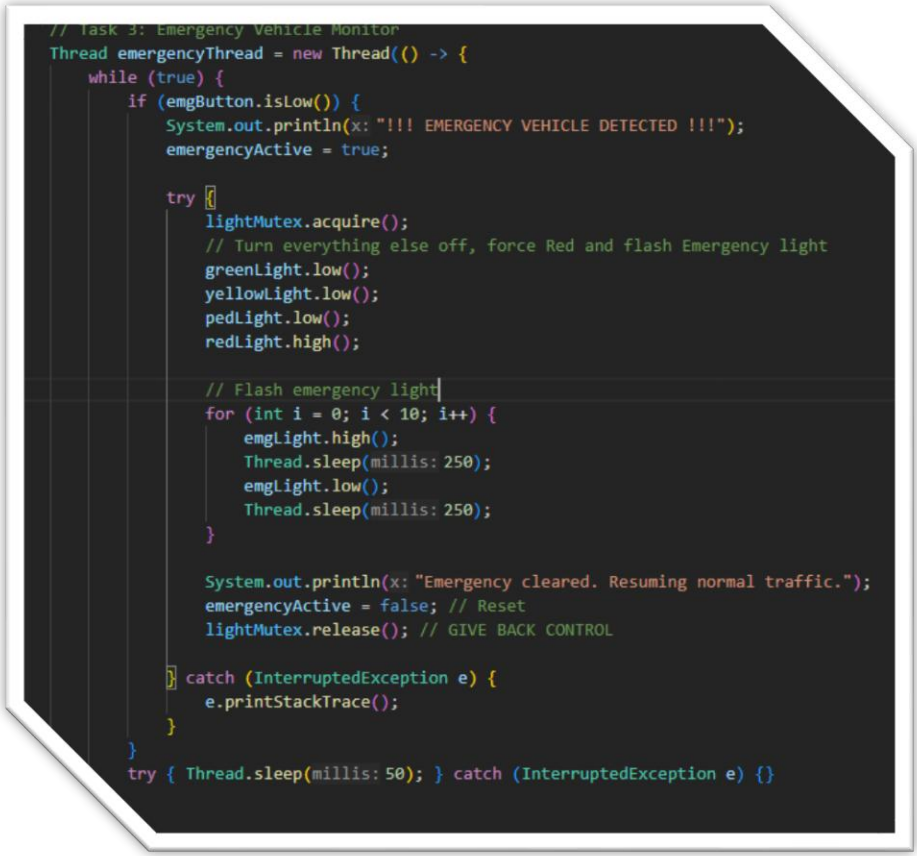
This thread registered pedestrian walk requests. The task of this thread is to consecutively poll the first push button which turns the boolean pedestrianWaiting variable to true. The traffic threat reads this variable later when the light turns red.



```
// Task 2: Pedestrian Button Monitor
Thread pedestrianThread = new Thread(() -> {
    while (true) {
        if (pedButton.isLow() && !emergencyActive) {
            System.out.println(x: "Pedestrian button pressed!");
            pedestrianWaiting = true;
            try { Thread.sleep(millis: 1000); } catch (InterruptedException e) {}
        }
        try { Thread.sleep(millis: 50); } catch (InterruptedException e) {}
    }
);
```

Task3: Emergency Vehicle Thread

This thread detects the emergency vehicle and immediately overrides all intersection logic. It basically polls the second push button When it is trigger it sets emergencyActive variable to true which does the traffic thread to pause. Then it forces the traffic light to red and flashes the emergency white LED light.



```
// Task 3: Emergency Vehicle Monitor
Thread emergencyThread = new Thread(() -> {
    while (true) {
        if (emgButton.isLow()) {
            System.out.println(x: "!!! EMERGENCY VEHICLE DETECTED !!!");
            emergencyActive = true;

            try {
                lightMutex.acquire();
                // Turn everything else off, force Red and flash Emergency light
                greenLight.low();
                yellowLight.low();
                pedLight.low();
                redLight.high();

                // Flash emergency light
                for (int i = 0; i < 10; i++) {
                    emgLight.high();
                    Thread.sleep(millis: 250);
                    emgLight.low();
                    Thread.sleep(millis: 250);
                }

                System.out.println(x: "Emergency cleared. Resuming normal traffic.");
                emergencyActive = false; // Reset
                lightMutex.release(); // GIVE BACK CONTROL

            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        try { Thread.sleep(millis: 50); } catch (InterruptedException e) {}
```

RESOURCE MANAGEMENT AND CONCURRENCY ANALYSIS

To ensure the emergency vehicle preemption, we utilized threat priorities.

trafficThread.setPriority(Thread.NORM_PRIORITY)

pedestrianThread.setPriority(Thread.NORM_PRIORITY + 1)

emergencyThread.setPriority(Thread.MAX_PRIORITY)

By assigning maximum priority to the emergency the Java scheduler ensures that CPU is effectively allocated to the emergency sequence As soon as the pushbutton is pressed which overrides the traffic delay.

Similarly for this project the most critical and tough operating system concept to implement was semaphore which helped as a mutex for the GPIO hardware. It was initialized using:

private static final Semaphore lightMutex = new Semaphore(1);

To implement this we intended the physical LEDs to be shared resources. If the traffic thread tries to turn on the green light at the exact time the emergency thread is trying to turn the red light on the system encounters the race condition which can cause a crash. To avoid this we used ***lightMutex.acquire()*** which locks the lights. So if the emergency thread has acquired this semaphore the traffic thread is entirely blocked from changing any LED states until the emergency thread calls ***lightMutex.release().*** This operation ensured a transition that we intended.

Conclusion

In order to conclude, our traffic light successfully demonstrated core OS concepts. Multi threading monitoring of multiple inputs at a time. The implementation of thread priorities ensured that high priority tasks were executed without delay and our summer fall prevented a race condition proving resource management and concurrency are vital for controlling shared resources.